SOEN 6011 — Deliverable 2

Function F6: The Real Beta Function $B(x, y)$
From-Scratch Implementation, Tkinter GUI, Repository, and Updated Requirements

Shoban Bhowmik (Student ID 40325778)

2026-07-24 (version 0.2.0)

Abstract

This report documents Deliverable 2 for Function F6, the real Beta function $B(x, y) = \Gamma(x)\Gamma(y)/\Gamma(x + y)$ scoped to $x > 0$, $y > 0$. Problem 5 reimplements the function *from scratch*: the three elementary primitives the Deliverable 1 prototype borrowed from Python's `math` module (ln, exp, sin, and the constant π) are rebuilt using only arithmetic and control flow, via range reduction and convergent series, and the result is wrapped in a Tkinter graphical interface with a typed exception hierarchy and helpful error messages. Problem 6 places the source in a public Git repository with an incremental, high-quality commit history and a README. Problem 7 evolves the requirements baseline from v0.1 to v0.2 on the basis of the implemented behaviour. Every problem was supported by a documented CASTROFF generative-AI prompt whose output was critically evaluated and independently verified. A re-runnable harness passes **27/27** checks; the from-scratch build reproduces the D1 accuracy *exactly*, with a worst-case relative error of 1.17×10^{-14}, eight orders of magnitude inside the 10^{-6} tolerance.

Contents

1 Introduction and scope **1**

2 Problem 5 — From-scratch implementation and GUI **2**
2.1 Freezing the "from scratch" boundary 2
2.2 The elementary functions 2
2.3 Architecture and exception handling 3
2.4 The Tkinter GUI 3
2.5 Verification 3
2.6 GAI use and critical evaluation (Problem 5) 5

3 Problem 6 — Repository, commits, and README **5**

4 Problem 7 — Updated requirements (v0.1 → v0.2) **5**

5 Known limitations (honest disclosure) **5**

6 Decision rationale summary **6**

1 Introduction and scope

Deliverable 2 modifies the Deliverable 1 prototype rather than starting over (the project is iterative and incremental). D1 selected **Algorithm B** — the Gamma identity $B(x, y) = \Gamma(x)\Gamma(y)/\Gamma(x+y)$ evaluated in the log domain, $B = \exp(\ln\Gamma(x) + \ln\Gamma(y) - \ln\Gamma(x+y))$, with a

hand-written Lanczos $\ln\Gamma$ [1, 2] — and shipped a command-line prototype (`src/cli.py`) that borrowed only four elementary primitives from `math`. This deliverable removes that dependency (Problem 5), adds a Tkinter GUI (Problem 5), publishes the code (Problem 6), and updates the requirements (Problem 7). The supported domain remains $x > 0$, $y > 0$ (decision D-001); reference definitions are from the NIST DLMF [3] and Abramowitz & Stegun [4].

2 Problem 5 — From-scratch implementation and GUI

2.1 Freezing the "from scratch" boundary

The Problem 5 rule exempts *input, output, arithmetic, and user-interface* facilities and prohibits every other built-in or library function. The dependency inventory of the D1 prototype identified exactly four mathematical borrowings; Table 1 classifies them and names their from-scratch replacements (decision D-008). No `math.gamma`, `math.lgamma`, or library `beta` was ever used — those would defeat the exercise.

Table 1: From-scratch boundary: prohibited primitives and their replacements.

Borrowed (D1)	**Role**	**D2 replacement (`elementary.py`)**
`math.log`	natural logarithm	`ln` — mantissa reduction $x = m \cdot 2^e$ + atanh series
`math.exp`	exponential	`exp` — reduction $x = k \ln 2 + r$ + Taylor series
`math.sin`	reflection only	`sin` — argument reduction + Taylor series
`math.pi`	constant π	`PI` — correctly-rounded double literal
`math.isfinite`	overflow guard	`is_finite` — IEEE comparison (not maths)

Retained facilities are arithmetic operators, the `int`/`float` casts (numeric conversion/parsing), string formatting (output), Tkinter (UI), and exceptions (explicitly required). The harness check **IMPL-01** scans every shipped module and fails if any imports `math`, `numpy`, `scipy`, `mpmath`, or `cmath`; it passes.

2.2 The elementary functions

Each primitive follows the standard recipe used by production math libraries [5, 6]: *range reduction* to a small interval, a *rapidly convergent series*, then *reconstruction*. Every loop terminates in a bounded number of steps (REL-03): the series stop when the next term falls below the working precision, and the power-of-two scaling exits as soon as the magnitude leaves the double range. The logarithm is representative:

```
def ln(x):
    if x <= 0.0:
        raise ValueError("ln␣is␣defined␣only␣for␣x␣>␣0")
    e = 0; m = x                        # reduce mantissa into [1/sqrt2, sqrt2)
    while m >= _SQRT2:   m *= 0.5; e += 1
    while m < _SQRT1_2:  m *= 2.0; e -= 1
    s = (m - 1.0) / (m + 1.0)           # |s| <= 0.1716 -> fast series
    s2 = s * s; term = s; total = s; denom = 1
    while True:                          # ln(m) = 2*(s + s^3/3 + s^5/5 + ...)
        term *= s2; denom += 2
        delta = term / denom; total += delta
        if abs(delta) < _EPS * abs(total) + _EPS:
```

```
            break
    return e * LN2 + 2.0 * total       # ln(x) = e*ln2 + ln(m)
```

Verified against a trusted oracle over wide ranges, the primitives reach machine precision: worst relative error $\leq 6 \times 10^{-15}$ for exp, $\leq 6 \times 10^{-16}$ for ln, and worst absolute error $\leq 9 \times 10^{-16}$ for sin (harness check ACC-03p). Full derivations and pseudocode for all three are in `docs/algorithms/elementary_functions.md`.

2.3 Architecture and exception handling

The source is separated so the numerical core is testable without a GUI and reusable by both front-ends: `elementary.py` (primitives) → `beta_core.py` (pure $\ln\Gamma/\beta$, imports only `elementary`) → `validation.py` (parsing + messages) and `gui.py` (Tkinter). Errors use a typed hierarchy (decision D-011): a base `BetaError` with `InputError` (`Empty`/`NonNumeric`/`NonFinite`), `DomainError`, `RangeError`, and `NumericalError` (`NumericalRange`/`Convergence`) subclasses, each carrying a user-facing message. This lets the GUI catch input errors and numerical errors separately (ERR-02) while a single `except BetaError` still guarantees no traceback reaches the user (REL-01). The core signals overflow explicitly:

```
def beta(x, y):
    if x <= DOMAIN_MIN or y <= DOMAIN_MIN:
        raise DomainError("... supported for x > 0 and y > 0 ...")
    result = el.exp(beta_ln(x, y))              # log domain avoids overflow
    if not el.is_finite(result):
        raise NumericalRangeError("... too large or too small ...")
    return result
```

2.4 The Tkinter GUI

The interface was wireframed before coding (decision D-010, `docs/gui_wireframe.md`) and traces to the persona "Maya Fernandes": a single resizable window with two labelled fields that state the domain at the point of input ("`x (must be > 0)`"), Calculate/Clear/ Help buttons, and one status line. It is keyboard-first (UI-01): initial focus on x; Tab order $x \to y \to$ Calculate → Clear; Enter computes; Escape clears; F1 opens Help. Status meaning is carried by a leading word and a glyph, with colour only as a redundant cue (ACC-03). Figures 1–2 show a valid computation and an out-of-domain error; each expected failure produces a specific message, never a traceback (REL-01, ERR-01).

Note on figures. A live screen grab of a Tk window is blocked in a headless session, so Figures 1–2 are layout renders that reproduce the exact widgets and status strings emitted by `src/gui.py` (verified in `tests/verify_d2.py`, checks GUI-01–GUI-06); the genuine window is shown in the live Zoom demonstration.

2.5 Verification

The re-runnable harness `tests/verify_d2.py` passes **27/27** checks. It confirms the no-library boundary (IMPL-01); the primitive accuracy against `math` used *only as an oracle* (ACC-03p); the Beta accuracy across all 18 trusted reference values with a worst relative error of 1.17×10^{-14} including the $B(0.2, 0.3)$ corner (ACC-01); symmetry; the five validation paths each raising their specific typed exception with a helpful message (VAL/ERR); no non-finite output over 36 large/small inputs (REL-02); bounded work (REL-03); 7.8×10^{-3} ms per computation (PERF-01); a subprocess GUI launch (POR-01); and the six GUI event-handler behaviours. The requirement-by-requirement matrix is in `docs/verification_matrix_d2.md`.

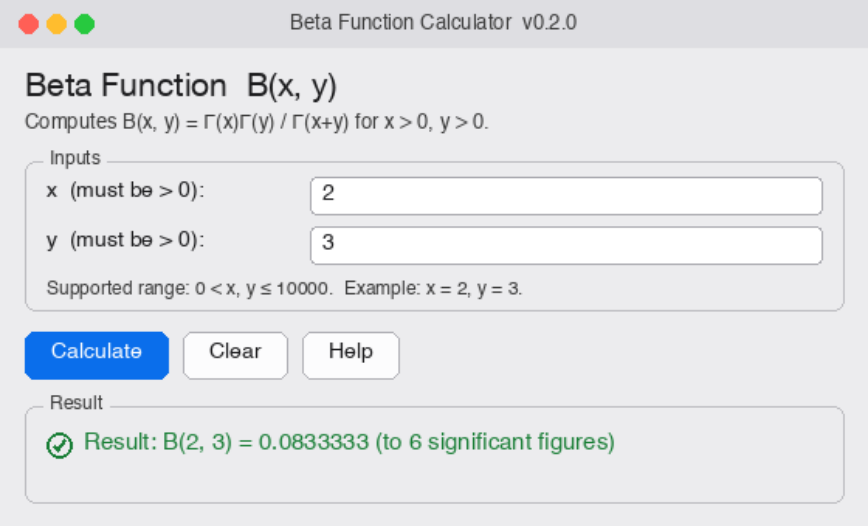

Figure 1: Valid case $B(2,3) = 0.0833333$ ($=1/12$), shown to six significant figures.

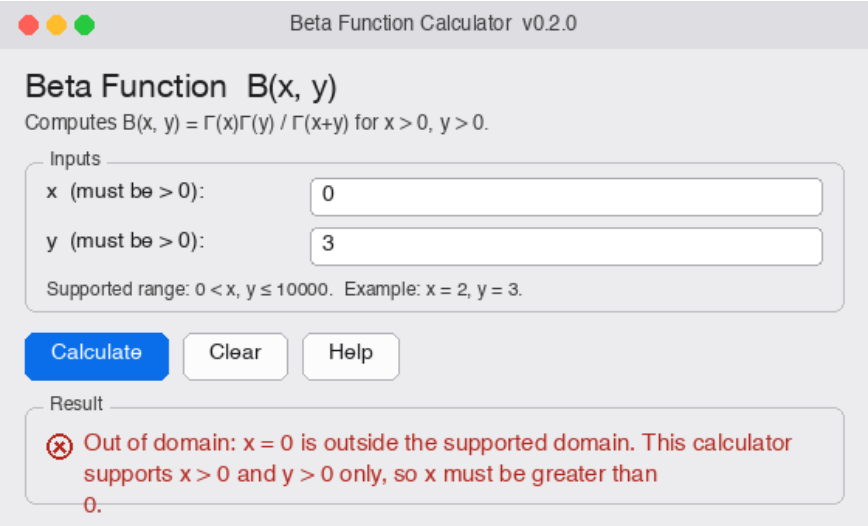

Figure 2: Out-of-domain input ($x = 0$): a specific, corrective message (ERR-01), conveyed as text and not by colour alone (ACC-03).

2.6 GAI use and critical evaluation (Problem 5)

Tool: Claude (Opus 4.8) via Claude Code. **Prompt type:** generative + transformational (refactor to from-scratch) with GUI design. The CASTROFF prompt supplied `src/cli.py` and the borrowed-primitive list, and required (i) a frozen boundary, (ii) from-scratch ln/exp/sin/π via range reduction + series while keeping the Lanczos $\ln\Gamma$, and (iii) a wireframed Tkinter GUI over a pure core with a typed exception hierarchy. **Accepted:** the reduction-plus-series design, the `BetaError` hierarchy, and the compute/UI separation. **Verified independently:** rather than trusting the output, the harness confirmed the shipped modules import no math library and that the primitives match `math` to $\leq 6 \times 10^{-15}$ and the Beta core reproduces the D1 accuracy exactly. **Adjusted:** floor/ round/finiteness were implemented by hand and 2^k by bounded repeated multiplication, so the boundary is honest. **Rejected:** any use of `math`/`numpy`, CORDIC/minimax (unnecessary complexity, D-009), and colour-only status (ACC-03).

3 Problem 6 — Repository, commits, and README

The source is under Git with an *incremental* history (decision D-013): one cohesive change per commit, with imperative, specific subjects (e.g. "Add from-scratch elementary functions (ln, exp, sin)", "Add Tkinter GUI over the pure core"), separating scaffold, primitives, core, validation/exceptions, GUI, tests, and documentation rather than a single bulk upload, so the commit log shows how the project evolved. A `.gitignore` excludes byte-code, `.DS_Store`, LaTeX build artefacts, IDE settings, and submission zips. The `README.md` lets a fresh user run the program: purpose and definition, prerequisites, the exact command (`python3 src/gui.py`), usage examples with screenshots, an error/troubleshooting guide, the repository structure, the version, and attribution. The public host uses the author's own account; the repository URL is `https://github.com/shobanbhowmik2/beta-function-calculator` and is recorded in the README on publication. Commit subjects and the publish commands are listed in the README and the D2 execution steps.

4 Problem 7 — Updated requirements (v0.1 → v0.2)

The requirements were compared line-by-line with the actual D2 behaviour and re-frozen as baseline **v0.2** (decision D-012). All v0.1 identifiers were retained for traceability; three requirements were added and several reworded for the GUI. Table 2 is the change log; the full v0.2 table (24 requirements) is in `docs/requirements/requirements_updated_v0.2.md`.
No requirement was deleted; superseded wording is preserved in v0.1. Assumptions A-04 (magnitude cap) and A-05 (tolerance) — flagged in D1 for re-validation — were confirmed against the from-scratch build, and A-07 was added for the elementary-function accuracy.

GAI use (Problems 6–7). Tool: Claude (Opus 4.8). For Problem 6 the CASTROFF prompt (advisory) produced the `.gitignore`, the incremental commit plan, and the README; the *publish* step was deliberately left to the author's own account (an outward-facing action), with exact commands provided. For Problem 7 the CASTROFF prompt (transformational + review) evolved the v0.1 baseline; every change was checked against a verification-matrix entry and a decision-log rationale, and superseded wording was preserved rather than deleted. Full logs: `docs/prompts/gai_prompt_log.md`.

5 Known limitations (honest disclosure)

The domain is $x, y > 0$ only (no analytic continuation or complex arguments); magnitudes above 10^4 are rejected by design; very large near-equal inputs (e.g. $B(10^4, 10^4)$) correctly underflow to

Table 2: Requirement changes v0.1 → v0.2 (all changes discovered through implementation unless noted).

Requirement	Change and reason
FR-01/03/04/05, USE-01/02	CLI prompts/commands → GUI labels/affordances (UI moved to Tkinter).
VAL-02	now also rejects non-finite text (`inf`/`nan`), which `float()` would otherwise accept.
ACC-01, A-05	re-scoped to the from-scratch build and re-validated (worst rel. err. unchanged at 1.17×10^{-14}).
ACC-03	priority S → M — accessibility is first-class for the GUI (design decision).
REL-01	strengthened: no traceback may reach the GUI; expected failures caught as typed `BetaErrors`.
REL-02	now raises an explicit `NumericalRangeError` instead of a silent guard.
POR-01	entry point `src/cli.py` → `src/gui.py`.
IMPL-01 (new)	the mathematics must be from scratch (Problem 5 mandate, D-008).
UI-01 (new)	the GUI must be fully keyboard-operable.
ERR-02 (new)	distinct exception types for input vs numerical errors.

the finite double 0.0; accuracy is bounded by the Lanczos g=7 set ($\approx$ 15 digits) and the series precision (verified $\leq 6 \times 10^{-15}$); and the GUI figures are renders, with the live window shown in the demo. Each is a documented scope boundary or assumption (D-001/D-004/D-007/D-008, A-01/A-04/A-05/A-06), not a correctness defect within the supported domain.

6 Decision rationale summary

D-008 freezes the from-scratch boundary; D-009 chooses range-reduction-plus-series for the primitives; D-010 fixes the wireframed GUI; D-011 defines the typed exception hierarchy; D-012 freezes requirements v0.2; D-013 sets the public-repo and commit strategy. All are recorded in `docs/decisions/decision_log.md`, and every GAI interaction is logged with its prompt, output summary, critical evaluation, and independent verification in `docs/prompts/gai_prompt_log.md`.

References

[1] C. Lanczos, "A precision approximation of the gamma function," *Journal of the Society for Industrial and Applied Mathematics, Series B: Numerical Analysis*, vol. 1, no. 1, pp. 86–96, 1964. Source of the Lanczos $\ln\Gamma$ approximation used in Algorithm B.

[2] W. H. Press, S. A. Teukolsky, W. T. Vetterling, and B. P. Flannery, *Numerical Recipes: The Art of Scientific Computing*. Cambridge University Press, 3rd ed., 2007. Adaptive quadrature (Algorithm A) and Lanczos coefficient set.

[3] NIST, "NIST Digital Library of Mathematical Functions, §5.12: Beta Function." `https://dlmf.nist.gov/5.12`, 2024. Beta function definitions and properties.

[4] M. Abramowitz and I. A. Stegun, *Handbook of Mathematical Functions with Formulas, Graphs, and Mathematical Tables*. U.S. National Bureau of Standards, 1964. Beta and Gamma function definitions and reference values.

[5] J.-M. Muller, *Elementary Functions: Algorithms and Implementation.* Birkhäuser, 3rd ed., 2016. Range reduction and series evaluation of exp, ln, and sin.

[6] W. J. Cody and W. M. Waite, "Software manual for the elementary functions," *Prentice-Hall Series in Computational Mathematics*, 1980. Argument range-reduction strategy for elementary functions.